

Two Pointers & Sliding Window — Pattern Guide

There are **4 core patterns** in this category. Master these and most array/string problems on this topic become recognizable.

Pattern 1: Constant (Fixed-Size) Window

Use case: Window size **k** is fixed. Slide it across the array by adding the new right element and removing the old left element.

Frequency in interviews: Low-to-medium — good warm-up pattern, rarely the "hard" question.

Example

Given `arr = [1, 2, 3, 4, 5, 6]`, find the maximum sum of any `k=3` consecutive numbers.

Answer: `[4, 5, 6]` → sum = 15

Code

```
int maxSumFixedWindow(vector<int>& arr, int k) {
    int n = arr.size();
    int windowSum = 0;

    // Build the first window
    for (int i = 0; i < k; i++) {
        windowSum += arr[i];
    }

    int maxSum = windowSum;

    // Slide the window: add arr[r], remove arr[r-k]
    for (int r = k; r < n; r++) {
        windowSum += arr[r] - arr[r - k];
        maxSum = max(maxSum, windowSum);
    }

    return maxSum;
}
```

Complexity: Time $O(N)$ · Space $O(1)$

★ Pattern 2: Longest Subarray / Substring where **<condition>**

Use case: Find the maximum length of a window that satisfies some condition. **Frequency in interviews:** Very high — this is the workhorse pattern.

Example

Find the length of the longest subarray with sum $\leq k$. $arr = [2, 5, 1, 7, 10]$, $k = 14$
→ subarray $[2, 5, 1]$ → length 3

(a) Brute Force — generate all subarrays

```
int longestSubarrayBrute(vector<int>& arr, int k) {
    int n = arr.size();
    int maxLen = 0;

    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = i; j < n; j++) {
            sum += arr[j];
            if (sum <= k) {
                maxLen = max(maxLen, j - i + 1);
            } else {
                break; // only valid if array has non-negative numbers
            }
        }
    }
    return maxLen;
}
```

Complexity: Time $O(N^2)$ · Space $O(1)$

(b) Optimal — Two Pointer + Sliding Window

```
int longestSubarrayOptimal(vector<int>& arr, int k) {
    int n = arr.size();
    int l = 0, sum = 0, maxLen = 0;

    for (int r = 0; r < n; r++) {
        sum += arr[r];
```

```

        // Shrink window while condition is violated
        while (sum > k) {
            sum -= arr[l];
            l++;
        }

        maxLen = max(maxLen, r - l + 1);
    }
    return maxLen;
}

```

Complexity: Time $O(N) + O(N) = O(2N)$ (each element added once by r , removed at most once by l) · Space $O(1)$

(c) More Optimized — when only LENGTH is needed (not the actual subarray)

If you don't need the exact subarray — just its length — you can skip the inner `while` and use `if` instead, checking the condition once per iteration. **Valid only when the array has non-negative numbers**, since the window size never needs to shrink below its previous maximum.

```

int longestSubarrayMostOptimal(vector<int>& arr, int k) {
    int n = arr.size();
    int l = 0, sum = 0, maxLen = 0;

    for (int r = 0; r < n; r++) {
        sum += arr[r];

        if (sum > k) {
            sum -= arr[l];
            l++;
        }

        maxLen = max(maxLen, r - l + 1);
    }
    return maxLen;
}

```

Complexity: Time $O(N)$ single pass · Space $O(1)$

Pattern 3: Number of Subarrays where `<condition>`

Use case: Count subarrays satisfying a condition. Solved by **reusing Pattern 2** via an `atMost(k)` helper. **Trick:** `countExactly(k) = atMost(k) - atMost(k - 1)`

Example

Number of subarrays with `sum == k` `arr = [1, 2, 1, 2]`, `k = 3` → subarrays `[1,2]`, `[2,1]` → answer **2**

Code

```
int atMost(vector<int>& arr, int k) {
    if (k < 0) return 0; // guard for k-1 going negative

    int n = arr.size();
    int l = 0, sum = 0, count = 0;

    for (int r = 0; r < n; r++) {
        sum += arr[r];

        while (sum > k) {
            sum -= arr[l];
            l++;
        }

        // every subarray ending at r, starting from l..r, is valid
        count += (r - l + 1);
    }
    return count;
}

int numberOfSubarraysWithSumK(vector<int>& arr, int k) {
    return atMost(arr, k) - atMost(arr, k - 1);
}
```

Complexity: Time $O(N)$ (two passes of `atMost`) · Space $O(1)$

Pattern 4: Shortest / Minimum Window where <condition>

Use case: Find the smallest window that satisfies a condition — shrink aggressively once valid.

Example

Find the length of the smallest subarray with sum $\geq k$. $arr = [2, 1, 5, 2, 3, 2]$, $k = 7 \rightarrow$ subarray $[5, 2] \rightarrow$ length **2**

Code

```
int shortestSubarray(vector<int>& arr, int k) {
    int n = arr.size();
    int l = 0, sum = 0;
    int minLen = INT_MAX;

    for (int r = 0; r < n; r++) {
        sum += arr[r];

        // Shrink from left WHILE window is still valid
        while (sum >= k) {
            minLen = min(minLen, r - l + 1);
            sum -= arr[l];
            l++;
        }
    }

    return (minLen == INT_MAX) ? 0 : minLen;
}
```

Complexity: Time $O(N)$ · Space $O(1)$

Quick Summary Table

Pattern	Goal	Technique	Time
1. Constant Window	Fixed size k	Slide by 1, add/remove edges	$O(N)$
2. Longest w/ condition	Maximize window length	Expand r , shrink l while invalid	$O(N)$
3. Count subarrays w/ condition	Count valid subarrays	Reuse Pattern 2 — $atMost(k) - atMost(k-1)$	$O(N)$
4. Shortest w/ condition	Minimize window length	Expand r , shrink l while still valid	$O(N)$

Recognition Cheat Sheet (how to spot which pattern to use)

- "exactly k elements / size k window" → Pattern 1
- "longest / maximum length subarray/substring satisfying X" → Pattern 2
- "how many subarrays satisfy X" → Pattern 3
- "shortest / minimum length subarray/substring satisfying X" → Pattern 4
- If the array has **negative numbers**, sliding window with a simple sum condition usually breaks — think prefix sum + hashmap instead.